

Agentic AI Chapter 2: Prompt Engineering, Tool Calling & Structured Outputs

Core Learning Objective: Move beyond simple text generation. Learn how LLMs interface with the external world through the Function Calling protocol, JSONSchema validation, and Pydantic v2 structured outputs.

1. The Prompt Engineering Hierarchy

flowchart TD

Prompts["Prompting Paradigms"]

Prompts → Zero["1. Zero-Shot
(Direct prompt without examples)"]

Prompts → Few["2. Few-Shot
(In-context input/output demonstrations)"]

Prompts → CoT["3. Chain-of-Thought (CoT)
('Let's think step by step' forces int

Prompts → Structured["4. Structured Schema Prompting
(Guaranteed JSON generatio

2. The Tool Calling (Function Calling) Protocol Under the Hood

Tool calling transforms an LLM from a passive text generator into an **autonomous computational agent**. The LLM itself does **not** execute the code; it acts as a **smart orchestrator** that **decides which function to call and formats valid JSON arguments**.

```
sequenceDiagram
    autonumber
    actor User
    participant Runtime as Agent Execution Runtime (Python)
    participant LLM as LLM API (GPT-4 / Claude / Gemini)
    participant Tool as Real World Tool (Weather API / DB)

    User->>Runtime: "What's the weather in Tokyo?"
    Note over Runtime: Injects tool schema (JSONSchema) into LLM system prompt
    Runtime->>LLM: Prompt + Available Tools: [get_weather(city: str)]
    Note over LLM: LLM decides: Tool needed! Emits tool_call object (not text)
    LLM->>Runtime: tool_call: get_weather({"city": "Tokyo"})
    Runtime->>Tool: Execute Python function: get_weather("Tokyo")
    Tool->>Runtime: Return: {"temp_c": 19.5, "condition": "Sunny"}
    Runtime->>LLM: Append Tool Result message to conversation history
    Note over LLM: Synthesizes final answer using tool result
    LLM->>Runtime: "The current weather in Tokyo is 19.5°C and sunny."
    Runtime->>User: Delivers final response
```

3. Production-Grade Tool Calling Implementation in Python

Here is a complete, runnable in-memory tool loop implementing function calling without third-party dependencies:

```

import json
import inspect
from typing import Callable, Any, Dict, List

# --- Tool Registry ---
class ToolRegistry:
    def __init__(self):
        self._tools: Dict[str, Callable] = {}
        self._schemas: List[Dict[str, Any]] = []

    def register(self, func: Callable):
        """Inspects Python function signature and auto-generates JSONSchema."""
        name = func.__name__
        doc = func.__doc__ or "No description provided."
        sig = inspect.signature(func)

        properties = {}
        required = []
        for param_name, param in sig.parameters.items():
            param_type = "string" if param.annotation == str else "number"
            properties[param_name] = {"type": param_type, "description": f"Parameter {param_name}"
            if param.default == inspect.Parameter.empty:
                required.append(param_name)

        schema = {
            "type": "function",
            "function": {
                "name": name,
                "description": doc.strip(),
                "parameters": {
                    "type": "object",
                    "properties": properties,
                    "required": required
                }
            }
        }

        self._tools[name] = func
        self._schemas.append(schema)

    def execute(self, tool_name: str, arguments_json: str) → str:
        if tool_name not in self._tools:

```

```

        return json.dumps({"error": f"Tool '{tool_name}' not found."})

    try:
        kwargs = json.loads(arguments_json)
        result = self._tools[tool_name](**kwargs)
        return json.dumps(result)
    except Exception as e:
        return json.dumps({"error": str(e)})

    @property
    def schemas(self) → List[Dict[str, Any]]:
        return self._schemas

# --- Define Native Tools ---
tools = ToolRegistry()

def get_stock_price(symbol: str) → dict:
    """Fetches real-time stock price for a given ticker symbol."""
    prices = {"AAPL": 225.50, "GOOGL": 178.20, "NVDA": 130.40}
    return {"symbol": symbol.upper(), "price": prices.get(symbol.upper(), "Ticker not found")}

tools.register(get_stock_price)

print("Generated Tool Schema:")
print(json.dumps(tools.schemas, indent=2))

# Simulate LLM tool execution:
simulated_llm_call = {
    "name": "get_stock_price",
    "arguments": '{"symbol": "NVDA"}'
}

tool_output = tools.execute(simulated_llm_call["name"], simulated_llm_call["arguments"])
print("\nTool Execution Output:", tool_output)

```

4. Structured Output Enforcement with Pydantic v2

Modern enterprise systems forbid free-form text when extracting entities or generating database records. Pydantic v2 guarantees schema adherence:

```
from pydantic import BaseModel, Field
from typing import List

class ExtractedEntities(BaseModel):
    company_name: str = Field(description="Name of the enterprise")
    quarter: str = Field(description="Fiscal quarter e.g. Q3 2026")
    revenue_billions: float = Field(description="Reported revenue in billions USD")
    risks_identified: List[str] = Field(description="Bullet list of identified risks")

# Models configured with response_format={"type": "json_object"} enforce
# that output conforms 100% to ExtractedEntities.model_json_schema()!
print("\nPydantic Validation JSON Schema:")
print(json.dumps(ExtractedEntities.model_json_schema(), indent=2))
```